NP-schwer und NP-vollständig

Marc Hellmuth

Universität Leipzig - 2025

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von . . . Problemen

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von . . . Problemen

- Es gibt Probleme, die man nicht in endlicher Zeit mit Algorithmen lösen kann (“unlösbar”)
  z.B. Halteproblem [prüfe ob ein geg. Alg. terminiert]

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von lösbaren Problemen

- Es gibt Probleme, die man nicht in endlicher Zeit mit Algorithmen lösen kann (“unlösbar”)
  z.B. Halteproblem [prüfe ob ein geg. Alg. terminiert]
- Es gibt Probleme, die man in endlicher Zeit (z.B. in Exponentialzeit $\mathcal{O}(1.5^n)$) mit Alg. lösen kann (“lösbar”)

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von lösbaren Problemen

- Es gibt Probleme, die man nicht in endlicher Zeit mit Algorithmen lösen kann (“unlösbar”)
  z.B. Halteproblem [prüfe ob ein geg. Alg. terminiert]
- Es gibt Probleme, die man in endlicher Zeit (z.B. in Exponentialzeit $\mathcal{O}(1.5^n)$) mit Alg. lösen kann (“lösbar”)
  Lösbare Probleme können unterteilt werden in

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von lösbaren Problemen

- Es gibt Probleme, die man nicht in endlicher Zeit mit Algorithmen lösen kann (“unlösbar”)
  z.B. Halteproblem [prüfe ob ein geg. Alg. terminiert]
- Es gibt Probleme, die man in endlicher Zeit (z.B. in Exponentialzeit $\mathcal{O}(1.5^n)$) mit Alg. lösen kann (“lösbar”)
  Lösbare Probleme können unterteilt werden in
  - Probleme die man mit Polynomialzeit Algorithmen lösen kann (“einfach”)

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von lösbaren Problemen

- Es gibt Probleme, die man nicht in endlicher Zeit mit Algorithmen lösen kann (“unlösbar”)
  z.B. Halteproblem [prüfe ob ein geg. Alg. terminiert]
- Es gibt Probleme, die man in endlicher Zeit (z.B. in Exponentialzeit $\mathcal{O}(1.5^n)$) mit Alg. lösen kann (“lösbar”)
  Lösbare Probleme können unterteilt werden in
  - Probleme die man mit Polynomialzeit Algorithmen lösen kann (“einfach”)
  - und in solche Probleme . . . mmh . . .

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von lösbaren Problemen

- Es gibt Probleme, die man nicht in endlicher Zeit mit Algorithmen lösen kann (“unlösbar”)
  z.B. Halteproblem [prüfe ob ein geg. Alg. terminiert]
- Es gibt Probleme, die man in endlicher Zeit (z.B. in Exponentialzeit $\mathcal{O}(1.5^n)$) mit Alg. lösen kann (“lösbar”)
  Lösbare Probleme können unterteilt werden in
  - Probleme die man mit Polynomialzeit Algorithmen lösen kann (“einfach”)
  - und in solche Probleme . . . mmh . . . Was ist mit den lösbaren Problemen für die bisher kein Polynomialzeit Algorithmus gefunden wurde?

Einfache vs. Schwere Probleme

Ziel: Klassifizierung von lösbaren Problemen

- Es gibt Probleme, die man nicht in endlicher Zeit mit Algorithmen lösen kann (“unlösbar”)
  z.B. Halteproblem [prüfe ob ein geg. Alg. terminiert]
- Es gibt Probleme, die man in endlicher Zeit (z.B. in Exponentialzeit $\mathcal{O}(1.5^n)$) mit Alg. lösen kann (“lösbar”)
  Lösbare Probleme können unterteilt werden in
  - Probleme die man mit Polynomialzeit Algorithmen lösen kann (“einfach”)
  - und in solche Probleme . . . mmh . . . Was ist mit den lösbaren Problemen für die bisher kein Polynomialzeit Algorithmus gefunden wurde?

    $\Rightarrow$ Sollten wir weiter nach Polynomialzeit Algorithmen suchen?

    Oder gibt es **gute Gründe** die Suche nach einem Polynomialzeit Algorithmus zu beenden?

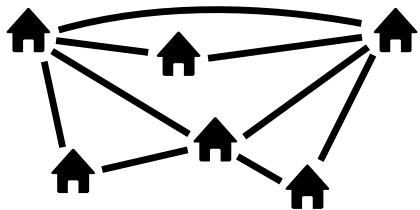
Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

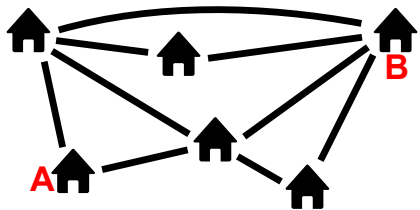
Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen



Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

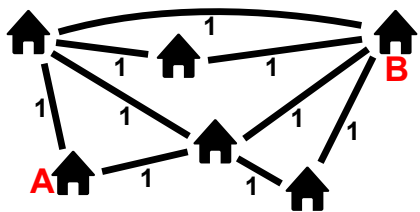


Finde einen kürzesten Pfad zwischen A und B unter der Annahme, dass alle Kanten e folgendes Gewicht haben:

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen



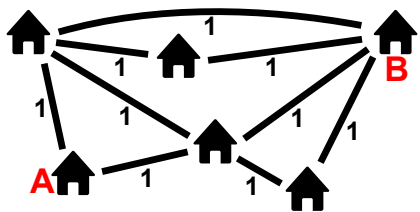
Finde einen kürzesten Pfad zwischen A und B unter der Annahme, dass alle Kanten e folgendes Gewicht haben:

$$w(e) = \mathbf{1}$$

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen



Finde einen kürzesten Pfad zwischen A und B unter der Annahme, dass alle Kanten e folgendes Gewicht haben:

$w(e) = \mathbf{1}$

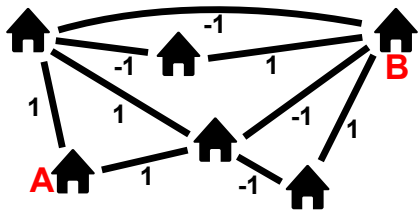
Lösbar mit Polynomialzeit Alg.

einfach!

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen



Finde einen kürzesten Pfad zwischen A und B unter der Annahme, dass alle Kanten e folgendes Gewicht haben:

$w(e) = \mathbf{1}$ $\qquad$ $w(e) \in \{\mathbf{1}, \mathbf{\text{-}1}\}$

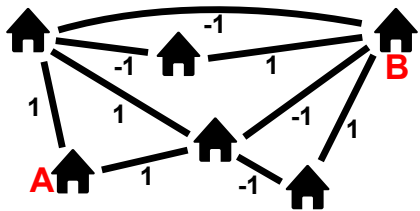
Lösbar mit Polynomialzeit Alg.

einfach!

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen



Finde einen kürzesten Pfad zwischen A und B unter der Annahme, dass alle Kanten e folgendes Gewicht haben:

$w(e) = \mathbf{1}$

Lösbar mit Polynomialzeit Alg.

einfach!

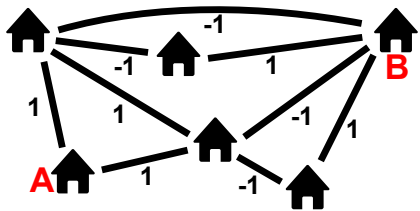
$w(e) \in \{\mathbf{1}, \mathbf{\text{-}1}\}$

Bisher sind nur Exponentialzeit Alg. bekannt

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen



Finde einen kürzesten Pfad zwischen A und B unter der Annahme, dass alle Kanten e folgendes Gewicht haben:

$w(e) = \mathbf{1}$

Lösbar mit Polynomialzeit Alg.

einfach!

$w(e) \in \{\mathbf{1}, \mathbf{\text{-}1}\}$

Bisher sind nur Exponentialzeit Alg. bekannt

Gibt es keine Poly.zeit Alg.? 🤔

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Boolesche Formel $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ mit Klauseln $C_1, \ldots, C_k$.

Finde eine Belegung der Variablen sd φ erfüllt wird, unter der Annahme, dass

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Boolesche Formel $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ mit Klauseln $C_1, \ldots, C_k$.

Finde eine Belegung der Variablen sd φ erfüllt wird, unter der Annahme, dass

$|C_i| = 2,\ 1 \leq i \leq k$

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Boolesche Formel $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ mit Klauseln $C_1, \ldots, C_k$.

Finde eine Belegung der Variablen sd φ erfüllt wird, unter der Annahme, dass

$|C_i| = 2$, $1 \leq i \leq k$

Lösbar mit Polynomialzeit Alg.

einfach!

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Boolesche Formel $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ mit Klauseln $C_1, \ldots, C_k$.

Finde eine Belegung der Variablen sd φ erfüllt wird, unter der Annahme, dass

$|C_i| = 2,\ 1 \leq i \leq k$ $\qquad$ $|C_i| = 3,\ 1 \leq i \leq k$

Lösbar mit Polynomialzeit Alg.

einfach!

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Boolesche Formel $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ mit Klauseln $C_1, \ldots, C_k$.

Finde eine Belegung der Variablen sd φ erfüllt wird, unter der Annahme, dass

$\lvert C_i \rvert = 2,\ 1 \leq i \leq k$	$\lvert C_i \rvert = 3,\ 1 \leq i \leq k$
Lösbar mit Polynomialzeit Alg.	Bisher sind nur Exponentialzeit Alg. bekannt

einfach!

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Boolesche Formel $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ mit Klauseln $C_1, \ldots, C_k$.

Finde eine Belegung der Variablen sd φ erfüllt wird, unter der Annahme, dass

$\lvert C_i \rvert = 2,\ 1 \leq i \leq k$	$\lvert C_i \rvert = 3,\ 1 \leq i \leq k$
Lösbar mit Polynomialzeit Alg.	Bisher sind nur Exponentialzeit Alg. bekannt
einfach!	**Gibt es keine Poly.zeit Alg.?** 

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Boolesche Formel $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ mit Klauseln $C_1, \ldots, C_k$.

$\Rightarrow$ Sollten wir weiter nach Polynomialzeit Algorithmen suchen?

Oder gibt es **gute Gründe** die Suche nach einem Polynomialzeit Algorithmus zu beenden?

Einfach vs. Schwer ist nicht offensichtlich!

Wir sind daran interessiert (lösbare) Probleme zu klassifizieren in “einfach” vs “schwer”

Kleine Änderungen in der Problemformulierung können “einfache” Problem zu einem “schweren” machen

Es ist offenbar gar nicht so einfach, “leichte” von “schwierigen” Problemen zu unterscheiden.

(Q1) Was bedeutet **schwierig** formal?

(Q2) Gibt es Probleme, für die – *unter vernünftigen Annahmen* – kein Polynomialzeitalgorithmus existiert? Was genau sind diese Annahmen?

$\Rightarrow$ Sollten wir weiter nach Polynomialzeit Algorithmen suchen?

Oder gibt es **gute Gründe** die Suche nach einem Polynomialzeit Algorithmus zu beenden?

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Warum Entscheidungsprobleme anstatt Optimierungsprobleme?

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Warum Entscheidungsprobleme anstatt Optimierungsprobleme?

- Optimierungsprobleme lassen sie sich in Entscheidungsprobleme umwandeln – und umgekehrt.

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Warum Entscheidungsprobleme anstatt Optimierungsprobleme?

- Optimierungsprobleme lassen sie sich in Entscheidungsprobleme umwandeln – und umgekehrt.

Beispiel: *Kürzeste Pfade*

Optimierung: Finde den kürzesten Pfad.

Entscheidung: Gibt es einen Pfad mit Länge $\leq k$?

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Warum Entscheidungsprobleme anstatt Optimierungsprobleme?

- Optimierungsprobleme lassen sie sich in Entscheidungsprobleme umwandeln – und umgekehrt.

  Beispiel: *Kürzeste Pfade*

  Optimierung: Finde den kürzesten Pfad.

  Entscheidung: Gibt es einen Pfad mit Länge $\leq k$?

- Formal lassen sich Entscheidungsprobleme als *Sprache* auffassen, welches zusammen mit *Turing Maschinen* eine formale Definition des Ausdrucks *“Ein Algorithmus löst ein Problem”* erlaubt.

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Warum Entscheidungsprobleme anstatt Optimierungsprobleme?

- Optimierungsprobleme lassen sie sich in Entscheidungsprobleme umwandeln – und umgekehrt.

  Beispiel: *Kürzeste Pfade*

  Optimierung: Finde den kürzesten Pfad.

  Entscheidung: Gibt es einen Pfad mit Länge $\leq k$?

- Formal lassen sich Entscheidungsprobleme als *Sprache* auffassen, welches zusammen mit *Turing Maschinen* eine formale Definition des Ausdrucks *“Ein Algorithmus löst ein Problem”* erlaubt.

- Um ”Schwere” eines Optimierungsproblems zu testen, genügt es, das dazugehörige Entscheidungsproblem zu betrachten.

  Eselsbrücke: Opt.-Problem in Polynomialzeit lösbar $\Rightarrow$ Ent.-Problem in Polynomialzeit lösbar
  Ent.-Problem *nicht* in Polynomialzeit lösbar $\Rightarrow$ Opt.-Problem *nicht* in Polynomialzeit lösbar

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

Die Klassen P und NP

Die Klassen P und NP sind mittels Turing Maschinen und Sprachen definiert

Wiederholung:

Für eine TM M ist $L(M) = \{w \in \Sigma^* \mid M \text{ akzeptiert } w\}$.

Eine Sprache L ist entscheidbar / lösbar, wenn es eine TM M gibt mit $L(M) = L$ und M auf jeder Eingabe hält. (D. h. M ist ein Entscheider für L; wir sagen: M löst L.)

Ein Entscheidungsproblem D kann mit der Sprache L_D seiner JA-Instanzen identifiziert werden. Wir nehmen dabei an, dass Instanzen I durch Wörter $w_I \in \Sigma^*$ effizient, insbesondere mit polynomialer Länge, codiert werden können.

Die Laufzeit eines Entscheiders M ist im Worst Case $T_M(n) = \max_{|w|=n}\{\text{Anzahl der Schritte von } M \text{ auf } w\}$.

Wir verwenden die Begriffe Algorithmus und TM im Folgenden synonym.

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\}$$

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\mathrm{NP} = \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\}$$

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\mathrm{NP} = \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\}$$

Wir haben nicht-deterministische TMs behandelt (mehrere nicht eindtg Übergange / ”parallel”)
Wie kann man sich das “besser” vorstellen?

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\mathrm{NP} = \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\}$$

Wir haben nicht-deterministische TMs behandelt (mehrere nicht eindtg Übergange / ”parallel”)
Wie kann man sich das “besser” vorstellen?

Ein nicht-deterministischer Alg./TM besteht aus 2 unabhängigen Schritten:

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\mathrm{NP} = \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\}$$

Wir haben nicht-deterministische TMs behandelt (mehrere nicht eindtg Übergange / ”parallel”)
Wie kann man sich das “besser” vorstellen?

Ein nicht-deterministischer Alg./TM besteht aus 2 unabhängigen Schritten:



Rate Schritt [*Rate eine Lösung*]

nicht-deterministische Teil: Wenn es ein JA-Antwort gibt, dann wird auch eine dieser JA-Antworten gefunden und zurück gegeben!

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\mathrm{NP} = \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\}$$

Wir haben nicht-deterministische TMs behandelt (mehrere nicht eindtg Übergange / ”parallel”)
Wie kann man sich das “besser” vorstellen?

Ein nicht-deterministischer Alg./TM besteht aus 2 unabhängigen Schritten:



Rate Schritt [*Rate eine Lösung*]
nicht-deterministische Teil: Wenn es ein JA-Antwort gibt, dann wird auch eine dieser JA-Antworten gefunden und zurück gegeben!

Checking Schritt [*Verifiziere, dass die Lsg. eine JA-antwort ist*]
in deterministischer Art und Weise.

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Wir haben nicht-deterministische TMs behandelt (mehrere nicht eindtg Übergange / ”parallel”)
Wie kann man sich das “besser” vorstellen?

Ein nicht-deterministischer Alg./TM besteht aus 2 unabhängigen Schritten:



Rate Schritt [*Rate eine Lösung*]
nicht-deterministische Teil: Wenn es ein JA-Antwort gibt, dann wird auch eine dieser JA-Antworten gefunden und zurück gegeben!

Checking Schritt [*Verifiziere, dass die Lsg. eine JA-antwort ist*]
in deterministischer Art und Weise.

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Wir haben nicht-deterministische TMs behandelt (mehrere nicht eindtg Übergange / ”parallel”)
Wie kann man sich das “besser” vorstellen?

Ein nicht-deterministischer Alg./TM besteht aus 2 unabhängigen Schritten:



Rate Schritt [*Rate eine Lösung*]
nicht-deterministische Teil: Wenn es ein JA-Antwort gibt, dann wird auch eine dieser JA-Antworten gefunden und zurück gegeben!

Checking Schritt [*Verifiziere, dass die Lsg. eine JA-antwort ist*]
in deterministischer Art und Weise.

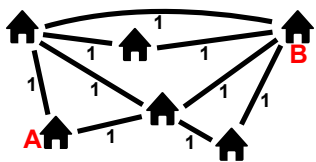
NP heisst “**N**icht-deterministisch **P**olynomial” (**NOT:** *nicht-polynomial*!)

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: Kürzeste Pfade



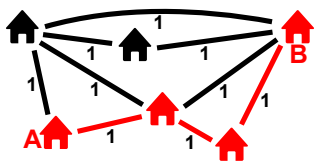
Frage: Gibt es einen Pfad der Länge≤ 4 von A nach B?

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: Kürzeste Pfade



Frage: Gibt es einen Pfad der Länge≤ 4 von A nach B?

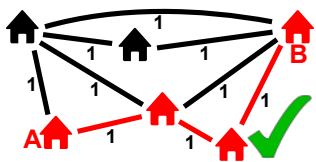
Rate Schritt [*Rate einmal eine Lsg*]

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: Kürzeste Pfade



Frage: Gibt es einen Pfad der Länge≤ 4 von A nach B?

Rate Schritt [*Rate einmal eine Lsg*]

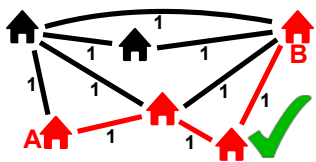
Checking Schritt [*Verifiziere ob Lsg eine JA-Antwort ist*]

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: Kürzeste Pfade $\in$ NP



Frage: Gibt es einen Pfad der Länge≤ 4 von A nach B?

Rate Schritt [*Rate einmal eine Lsg*]

Checking Schritt [*Verifiziere ob Lsg eine JA-Antwort ist*]

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: SAT

Input: Boolesche Formel

$\varphi = (x \vee y) \wedge (\neg x \vee \neg y \vee z)$

Frage: Gibt es Belegung der Variablen sd φ wahr wird?

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: SAT

Input: Boolesche Formel

$\varphi = (x \vee y) \wedge (\neg x \vee \neg y \vee z)$

Frage: Gibt es Belegung der Variablen sd φ wahr wird?

Rate Schritt [*Rate einmal eine Lsg.*]

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: SAT

Input: Boolesche Formel

$\varphi = (x \vee y) \wedge (\neg x \vee \neg y \vee z)$

Frage: Gibt es Belegung der Variablen sd φ wahr wird?

$x = 1, y = 0, z = 0$

Rate Schritt [*Rate einmal eine Lsg.*]

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: SAT

Input: Boolesche Formel

$\varphi = (x \vee y) \wedge (\neg x \vee \neg y \vee z)$

Frage: Gibt es Belegung der Variablen sd φ wahr wird?

$x = 1, y = 0, z = 0$

Rate Schritt [*Rate einmal eine Lsg.*]

Checking Schritt [*Verifiziere ob Lsg eine JA-Antwort ist*]

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: SAT

Input: Boolesche Formel

$\varphi = (x \vee y) \wedge (\neg x \vee \neg y \vee z)$

Frage: Gibt es Belegung der Variablen sd φ wahr wird?

$x = 1, y = 0, z = 0$
$\varphi = (1 \vee 0) \wedge (0 \vee 1 \vee 0) = 1$ ✓

Rate Schritt [*Rate einmal eine Lsg.*]

Checking Schritt [*Verifiziere ob Lsg eine JA-Antwort ist*]

Die Klassen P und NP

$$\begin{aligned} \mathrm{P} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\} \end{aligned}$$

$$\begin{aligned} \mathrm{NP} &= \{\text{Entsch. probl. kann in Polynomialzeit mit } \textbf{nicht-determ.} \text{ TM. gelöst werden}\} \\ &= \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\} \end{aligned}$$

Bsp: SAT $\in$ NP

Input: Boolesche Formel

$\varphi = (x \vee y) \wedge (\neg x \vee \neg y \vee z)$

Frage: Gibt es Belegung der Variablen sd φ wahr wird?

$x = 1, y = 0, z = 0$
$\varphi = (1 \vee 0) \wedge (0 \vee 1 \vee 0) = 1$ ✓

Rate Schritt [*Rate einmal eine Lsg.*]

Checking Schritt [*Verifiziere ob Lsg eine JA-Antwort ist*]

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$

P-NP-Problem: $\mathrm{P} \subset \mathrm{NP}$ oder $\mathrm{P} = \mathrm{NP}$?



Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$



P-NP-Problem: $\mathrm{P} \subset \mathrm{NP}$ oder $\mathrm{P} = \mathrm{NP}$?

Wenn $\mathrm{P} = \mathrm{NP}$, dann ist das Verifizieren genauso einfach wie das Finden einer Lösung

(unwahrscheinlich, aber bisher ungelöstes Problem!)

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$



P-NP-Problem: $\mathrm{P} \subset \mathrm{NP}$ oder $\mathrm{P} = \mathrm{NP}$?

Wenn $\mathrm{P} = \mathrm{NP}$, dann ist das Verifizieren genauso einfach wie das Finden einer Lösung

(unwahrscheinlich, aber bisher ungelöstes Problem!)

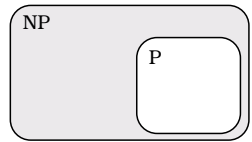
Annahme: $\mathrm{P} \subset \mathrm{NP}$

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$





P-NP-Problem: $\mathrm{P} \subset \mathrm{NP}$ oder $\mathrm{P} = \mathrm{NP}$?

Wenn $\mathrm{P} = \mathrm{NP}$, dann ist das Verifizieren genauso einfach wie das Finden einer Lösung

(unwahrscheinlich, aber bisher ungelöstes Problem!)

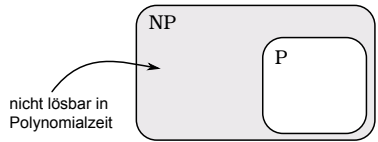
Annahme: $\mathrm{P} \subset \mathrm{NP}$

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$





P-NP-Problem: $\mathrm{P} \subset \mathrm{NP}$ oder $\mathrm{P} = \mathrm{NP}$?

Wenn $\mathrm{P} = \mathrm{NP}$, dann ist das Verifizieren genauso einfach wie das Finden einer Lösung

(unwahrscheinlich, aber bisher ungelöstes Problem!)

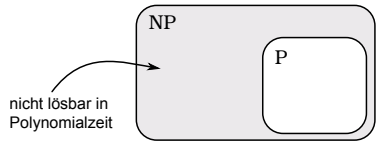
Annahme: $\mathrm{P} \subset \mathrm{NP}$

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$





P-NP-Problem: $\mathrm{P} \subset \mathrm{NP}$ oder $\mathrm{P} = \mathrm{NP}$?

Wenn $\mathrm{P} = \mathrm{NP}$, dann ist das Verifizieren genauso einfach wie das Finden einer Lösung

(unwahrscheinlich, aber bisher ungelöstes Problem!)

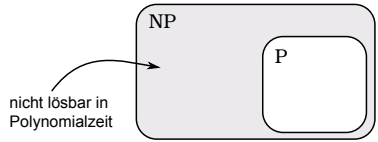
Annahme: $\mathrm{P} \subset \mathrm{NP} \implies$ Es gibt Probleme in NP die man nicht in Polynomialzeit lösen kann!

Die Klassen P und NP

$$\mathrm{P} = \{\text{Entsch. probl. } \textbf{lösbar} \text{ in Polynomialzeit}\}$$
$$\mathrm{NP} = \{\text{Entsch. probl. } \textbf{verifizierbar} \text{ in Polynomialzeit}\}$$

Beobachtung: $\mathrm{P} \subseteq \mathrm{NP}$





P-NP-Problem: $\mathrm{P} \subset \mathrm{NP}$ oder $\mathrm{P} = \mathrm{NP}$?

Wenn $\mathrm{P} = \mathrm{NP}$, dann ist das Verifizieren genauso einfach wie das Finden einer Lösung

(unwahrscheinlich, aber bisher ungelöstes Problem!)

Annahme: $\mathrm{P} \subset \mathrm{NP} \implies$ Es gibt Probleme in NP die man nicht in Polynomialzeit lösen kann!

(Q2) Gibt es Probleme, für die – *unter vernünftigen Annahmen* – kein Polynomialzeitalgorithmus existiert? Was genau sind diese Annahmen? Antwort: $\mathrm{P} \subset \mathrm{NP}$

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

$\mathrm{P} = \{\text{Entsch. probl. lösbar in Polynomialzeit}\}$
$\mathrm{NP} = \{\text{Entsch. probl. verifizierbar in Polynomialzeit}\}$

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

$\mathrm{P} = \{\text{Entsch. probl. lösbar in Polynomialzeit}\}$
$\mathrm{NP} = \{\text{Entsch. probl. verifizierbar in Polynomialzeit}\}$

Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.

Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



ARRR, wir setzen dich auf dieser einsamen Insel aus, wenn du die folgende Frage nicht innerhalb von 2 Minuten richtig beantwortest:

Sind weniger als 12.222 Münzen in der Truhe?

Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



ARRR, wir setzen dich auf dieser einsamen Insel aus, wenn du die folgende Frage nicht innerhalb von 2 Minuten richtig beantwortest:

Sind weniger als 12.222 Münzen in der Truhe?



Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



ARRR, wir setzen dich auf dieser einsamen Insel aus, wenn du die folgende Frage nicht innerhalb von 2 Minuten richtig beantwortest:

Sind weniger als 12.222 Münzen in der Truhe?



Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



ARRR, wir setzen dich auf dieser einsamen Insel aus, wenn du die folgende Frage nicht innerhalb von 2 Minuten richtig beantwortest:

Sind weniger als 12.222 Münzen in der Truhe?

DU: 1 Münze = 8g



Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



ARRR, wir setzen dich auf dieser einsamen Insel aus, wenn du die folgende Frage nicht innerhalb von 2 Minuten richtig beantwortest:

Sind weniger als 12.222 Münzen in der Truhe?

DU: 1 Münze = 8g
alle Münzen = 97760 g
= **12220** * 8g



Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



ARRR, wir setzen dich auf dieser einsamen Insel aus, wenn du die folgende Frage nicht innerhalb von 2 Minuten richtig beantwortest:

Sind weniger als 12.222 Münzen in der Truhe?

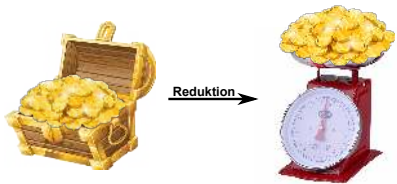
DU: 1 Münze = 8g
alle Münzen = 97760 g
= **12220** * 8g

Antwort: JAAAAA!!



Reduktion und NP-Schwere

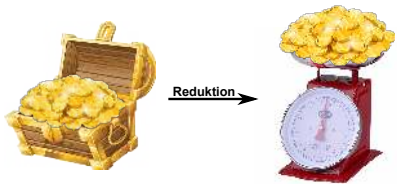
Reduktion = ein Problem in ein anderes Problem überführen.



Wir haben ein Zählproblem auf ein Wägeproblem reduziert!

Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.

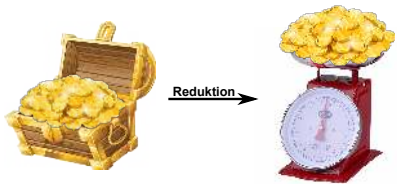


Wir haben ein Zählproblem auf ein Wägeproblem reduziert!

Dabei haben wir die “Einfachheit” des Wägeproblems ausgenutzt, um zu zeigen, dass das Zählproblem “einfach” ist.

Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



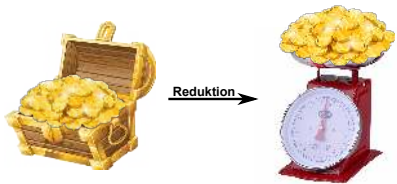
Wir haben ein Zählproblem auf ein Wägeproblem reduziert!

Dabei haben wir die “Einfachheit” des Wägeproblems ausgenutzt, um zu zeigen, dass das Zählproblem “einfach” ist.

Wir haben **auf** ein “einfaches” Problem reduziert

Reduktion und NP-Schwere

Reduktion = ein Problem in ein anderes Problem überführen.



Wir haben ein Zählproblem auf ein Wägeproblem reduziert!

Dabei haben wir die “Einfachheit” des Wägeproblems ausgenutzt, um zu zeigen, dass das Zählproblem “einfach” ist.

Wir haben **auf** ein “einfaches” Problem reduziert

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe.

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Ann.: Für D' gibt es keinen Polynomialzeit Alg.

D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Ann.: Für D' gibt es keinen Polynomialzeit Alg.

D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Kann es für D einen Polynomialzeit Alg. geben?

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Ann.: Für D' gibt es keinen Polynomialzeit Alg.

D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Kann es für D einen Polynomialzeit Alg. geben?

Beobachtung: Jede Instanz $I' = (M, X)$ kann zu einer Instanz $I = (M, X, Y)$ von D transformiert werden in dem man $X = Y$ setzt.

Instanz = *spezifizierte Eingabe* für das entsprechende Problem

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Ann.: Für D' gibt es keinen Polynomialzeit Alg.

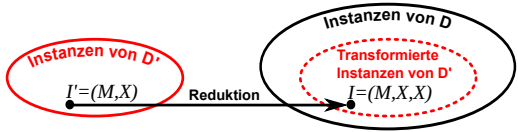
D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Kann es für D einen Polynomialzeit Alg. geben?

Beobachtung: Jede Instanz $I' = (M, X)$ kann zu einer Instanz $I = (M, X, Y)$ von D transformiert werden in dem man $X = Y$ setzt.

Instanz = *spezifizierte Eingabe* für das entsprechende Problem



Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Ann.: Für D' gibt es keinen Polynomialzeit Alg.

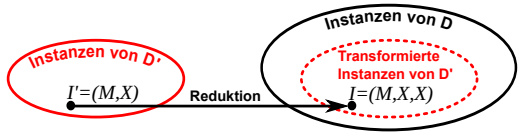
D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Kann es für D einen Polynomialzeit Alg. geben?

Beobachtung: Jede Instanz $I' = (M, X)$ kann zu einer Instanz $I = (M, X, Y)$ von D transformiert werden in dem man $X = Y$ setzt.

Instanz = *spezifizierte Eingabe* für das entsprechende Problem



Beispiel: Für $I' = (M, X = 9)$ wäre $I = (M, X = 9, Y = 9)$ eine Instanz von D

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Ann.: Für D' gibt es keinen Polynomialzeit Alg.

D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Kann es für D einen Polynomialzeit Alg. geben?

Beobachtung: Jede Instanz $I' = (M, X)$ kann zu einer Instanz $I = (M, X, Y)$ von D transformiert werden in dem man $X = Y$ setzt.

Instanz = *spezifizierte Eingabe* für das entsprechende Problem



$I' = (M, X)$ eine JA-Antwort $\iff$ $I = (M, X, X)$ eine JA-Antwort

Reduktion und NP-schwere

Um die “Schwere” eines Problems zu zeigen, reduzieren **von** einem “schweren” Problem.

Sie haben n Münzen wobei Münze i Gewicht w_i (=nat. Zahl) habe. Sei $M = \{w_1, \ldots, w_n\}$ die Menge der n Münzgewichte

D' (Teilsummen Problem)
IN: M & natürliche Zahl $X \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. $\sum_{w_i \in \tilde{M}} w_i = X$?

Beispiel: $I' = (M, X)$ für D' mit $M = \{2, 6, 7, 19, 100\}$ und $X = 9$.

Ann.: Für D' gibt es keinen Polynomialzeit Alg.

D (Einfaches Rucksackproblem))
IN: M & natürliche Zahlen $X, Y \geq 1$ **Q:** Gibt es eine Teilmenge $\tilde{M} \subseteq M$ s.d. s.d. $Y \leq \sum_{w_i \in \tilde{M}} w_i \leq X$?

$I = (M, X, Y)$ für D mit $M = \{2, 6, 7, 19, 100\}$ und $Y = 6$, $X = 10$

Kann es für D einen Polynomialzeit Alg. geben?

Beobachtung: Jede Instanz $I' = (M, X)$ kann zu einer Instanz $I = (M, X, Y)$ von D transformiert werden in dem man $X = Y$ setzt.

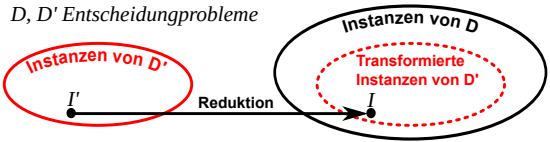
Instanz = *spezifizierte Eingabe* für das entsprechende Problem



$I' = (M, X)$ eine JA-Antwort $\iff$ $I = (M, X, X)$ eine JA-Antwort

$\implies$ **Für D gibt es keinen Polynomialzeit Alg.**

Reduktion und NP-schwere



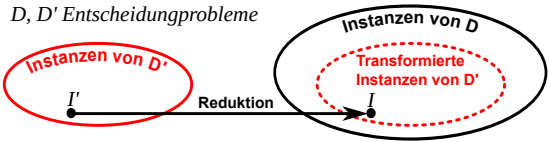
Instanz = *spezifizierte Eingabe.*

P = {Entsch.-Probl. lösbar in Polynomialzeit}

NP = {Entsch.Probl. verifizierbar in Polynomialzeit}

Annahme: $P \subset NP$

Reduktion und NP-schwere



Instanz = *spezifizierte Eingabe.*

$\mathrm{P} = \{\text{Entsch.-Probl. lösbar in Polynomialzeit}\}$

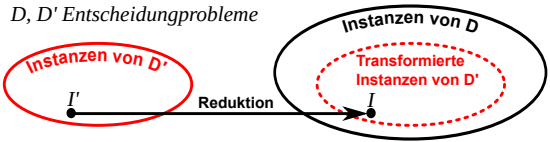
$\mathrm{NP} = \{\text{Entsch.Probl. verifizierbar in Polynomialzeit}\}$

Annahme: $P \subset NP$

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

Reduktion und NP-schwere



$\mathrm{P} = \{\text{Entsch.-Probl. lösbar in Polynomialzeit}\}$

$\mathrm{NP} = \{\text{Entsch.Probl. verifizierbar in Polynomialzeit}\}$

Annahme: $P \subset NP$

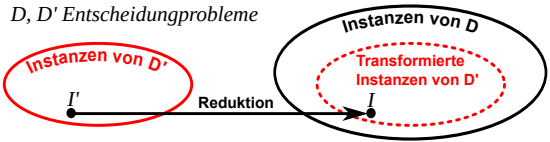
Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Reduktion und NP-schwere



$\mathrm{P} = \{\text{Entsch.-Probl. lösbar in Polynomialzeit}\}$

$\mathrm{NP} = \{\text{Entsch.Probl. verifizierbar in Polynomialzeit}\}$

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

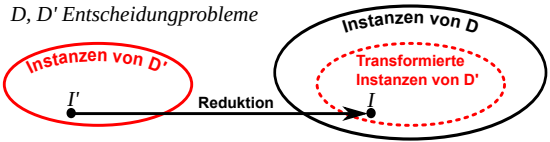
Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

Reduktion und NP-schwere



$\mathrm{P} = \{\text{Entsch.-Probl. lösbar in Polynomialzeit}\}$

$\mathrm{NP} = \{\text{Entsch.Probl. verifizierbar in Polynomialzeit}\}$

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

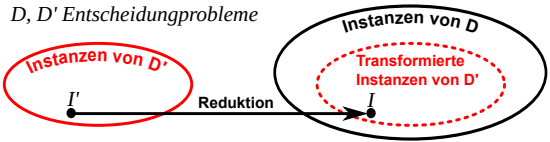
- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

Wenn D NP-schwer ist, dann

Reduktion und NP-schwere



$\mathrm{P} = \{\text{Entsch.-Probl. lösbar in Polynomialzeit}\}$

$\mathrm{NP} = \{\text{Entsch.Probl. verifizierbar in Polynomialzeit}\}$

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

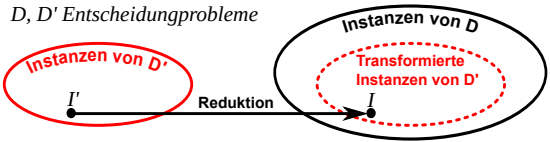
$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

Wenn D NP-schwer ist, dann

- kann jedes Problem $D' \in \mathrm{NP}$ als “Spezialfall” von D betrachtet werden.

Reduktion und NP-schwere



$\mathrm{P} = \{\text{Entsch.-Probl. lösbar in Polynomialzeit}\}$

$\mathrm{NP} = \{\text{Entsch.Probl. verifizierbar in Polynomialzeit}\}$

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

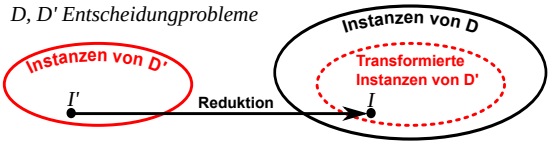
$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

Wenn D NP-schwer ist, dann

- kann jedes Problem $D' \in \mathrm{NP}$ als “Spezialfall” von D betrachtet werden.
- D ist mindestens so schwer zu lösen wie jedes andere Problem in NP.

Reduktion und NP-schwere



P = {Entsch.-Probl. lösbar in Polynomialzeit}

NP = {Entsch.Probl. verifizierbar in Polynomialzeit}

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

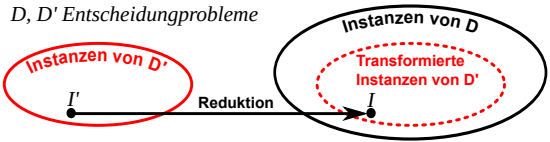
$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

Wenn D NP-schwer ist, dann

- kann jedes Problem $D' \in \mathrm{NP}$ als “Spezialfall” von D betrachtet werden.
- D ist mindestens so schwer zu lösen wie jedes andere Problem in NP.
- D ist mindestens so schwer zu lösen wie solche Probleme in NP für die es keinen Polynomialzeit Alg. gibt ($\mathrm{P} \subset \mathrm{NP}$!)

Reduktion und NP-schwere



P = {Entsch.-Probl. lösbar in Polynomialzeit}

NP = {Entsch.Probl. verifizierbar in Polynomialzeit}

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

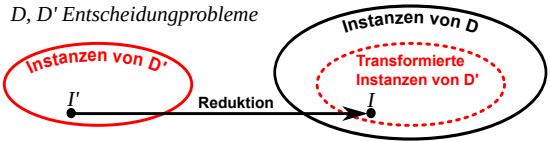
$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

Wenn D NP-schwer ist, dann

- kann jedes Problem $D' \in \mathrm{NP}$ als “Spezialfall” von D betrachtet werden.
- D ist mindestens so schwer zu lösen wie jedes andere Problem in NP.
- D ist mindestens so schwer zu lösen wie solche Probleme in NP für die es keinen Polynomialzeit Alg. gibt ($\mathrm{P} \subset \mathrm{NP}$!)
- Es gibt keinen Algorithmus der alle Instanzen von D in Polynomialzeit lösen kann

Reduktion und NP-schwere



P = {Entsch.-Probl. lösbar in Polynomialzeit}

NP = {Entsch.Probl. verifizierbar in Polynomialzeit}

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

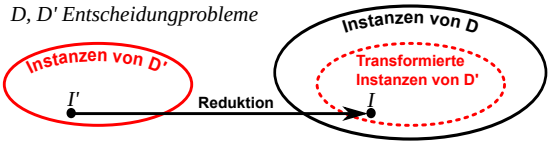
$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

Wenn D NP-schwer ist, dann

- kann jedes Problem $D' \in \mathrm{NP}$ als “Spezialfall” von D betrachtet werden.
- D ist mindestens so schwer zu lösen wie jedes andere Problem in NP.
- D ist mindestens so schwer zu lösen wie solche Probleme in NP für die es keinen Polynomialzeit Alg. gibt ($\mathrm{P} \subset \mathrm{NP}$!)
- Es gibt keinen Algorithmus der alle Instanzen von D in Polynomialzeit lösen kann
- Für NP-schwere Probleme dürfen wir die Suche nach allgemeinen Polynomialzeit Algorithmen aufgeben!

Reduktion und NP-schwere



$\mathrm{P} = \{\text{Entsch.-Probl. lösbar in Polynomialzeit}\}$

$\mathrm{NP} = \{\text{Entsch.Probl. verifizierbar in Polynomialzeit}\}$

Annahme: $P \subset NP$

Instanz = *spezifizierte Eingabe.*

Eine **Reduktion von D' auf D** ist eine Algorithmus welcher

- in Polynomialzeit
- jede Instanz I' von D' in eine Instanz I von D in transformiert, s.d.: I hat JA-Antwort $\iff$ I' hat JA-Antwort.

$\Rightarrow$ Jeder Algorithmus, der D löst, kann verwendet werden, um D' zu lösen.

Ein Entsch.-problem D ist **NP-schwer** wenn *jedes* Problem $D' \in \mathrm{NP}$ auf D reduziert werden kann.

(Q1) Was bedeutet **schwierig** formal? Antwort: NP-schwer
Bemerkung: Dies ist natürlich nur ein Weg ”schwierig” formal zu definieren.

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

$\mathrm{P} = \{\text{Entsch. probl. lösbar in Polynomialzeit}\}$
$\mathrm{NP} = \{\text{Entsch. probl. verifizierbar in Polynomialzeit}\}$

Einfach vs. Schwer ist nicht offensichtlich!

Zutaten:

1. Entscheidungsprobleme
   (Lösung ist entweder Ja oder Nein)
2. Klassen P und NP
3. Reduktion von Problemen und NP-schwere
4. NP-vollständigkeit

$\mathrm{P} = \{\text{Entsch. probl. lösbar in Polynomialzeit}\}$
$\mathrm{NP} = \{\text{Entsch. probl. verifizierbar in Polynomialzeit}\}$

NP-vollständigkeit

(Q1) Was bedeutet **schwierig** formal? Antwort: NP-schwer

(Q2) Gibt es Probleme, für die – *unter vernünftigen Annahmen* – kein Polynomialzeitalgorithmus existiert?

Was genau sind diese Annahmen? Antwort: $\mathrm{P} \neq \mathrm{NP}$

NP-vollständigkeit

(Q1) Was bedeutet **schwierig** formal? Antwort: NP-schwer

(Q2) Gibt es Probleme, für die – *unter vernünftigen Annahmen* – kein Polynomialzeitalgorithmus existiert?

Was genau sind diese Annahmen? Antwort: $\mathrm{P} \neq \mathrm{NP}$

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

NP-vollständigkeit

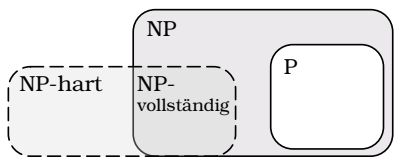
(Q1) Was bedeutet **schwierig** formal? Antwort: NP-schwer

(Q2) Gibt es Probleme, für die – *unter vernünftigen Annahmen* – kein Polynomialzeitalgorithmus existiert?

Was genau sind diese Annahmen? Antwort: $\mathrm{P} \neq \mathrm{NP}$

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP}\colon D' \preceq_p D$)



NP-vollständigkeit

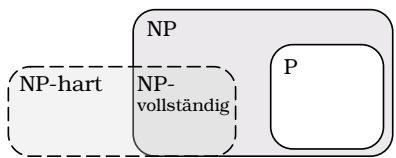
(Q1) Was bedeutet **schwierig** formal? Antwort: NP-schwer

(Q2) Gibt es Probleme, für die – *unter vernünftigen Annahmen* – kein Polynomialzeitalgorithmus existiert?
Ja, NP-vollständige Problem

Was genau sind diese Annahmen? Antwort: $\mathrm{P} \neq \mathrm{NP}$

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP}\colon D' \preceq_p D$)



NP-vollständigkeit

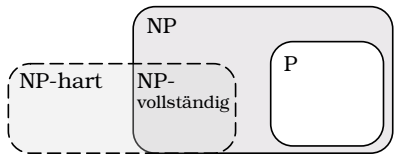
(Q1) Was bedeutet **schwierig** formal? Antwort: NP-schwer

(Q2) Gibt es Probleme, für die – *unter vernünftigen Annahmen* – kein Polynomialzeitalgorithmus existiert?
Ja, NP-vollständige Problem

Was genau sind diese Annahmen? Antwort: $\mathrm{P} \neq \mathrm{NP}$

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP}\colon D' \preceq_p D$)



Zusammenfassend:

- Falls $P \neq NP$, besitzt kein NP-vollständiges Problem einen Polynomialzeitalgorithmus.
- NP-vollständige Probleme sind die schwersten Probleme in NP.

NP-vollständigkeit

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

NP-vollständigkeit

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

Problem:

Q: Wie zeigt man, dass ALLE Problem in NP auf D reduziert werden können?

A: Es gab ein erstes Problem, nämlich SAT, von dem gezeigt wurde, dass es NP-vollständig ist.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$I \in L_D \iff \varphi_I \text{ ist erfüllbar.}$$

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Ein Berechnungspfad von M ist eine Folge von Konfigurationen $C_0 \to C_1 \to \cdots \to C_T$, wobei jeder Übergang durch die Übergangsfunktion von M erlaubt ist.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Ein Berechnungspfad von M ist eine Folge von Konfigurationen $C_0 \to C_1 \to \cdots \to C_T$, wobei jeder Übergang durch die Übergangsfunktion von M erlaubt ist.

Da M in Polynomialzeit läuft, gibt es ein Polynom p mit $T \leq p(|I|)$ für jeden Berechnungspfad auf Eingabe I.
Dabei gilt

$$I \in L_D \iff \text{es existiert ein akzeptierender Berechnungspfad von } M \text{ auf } I.$$

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Ein Berechnungspfad von M ist eine Folge von Konfigurationen $C_0 \to C_1 \to \cdots \to C_T$, wobei jeder Übergang durch die Übergangsfunktion von M erlaubt ist.

Da M in Polynomialzeit läuft, gibt es ein Polynom p mit $T \leq p(|I|)$ für jeden Berechnungspfad auf Eingabe I.
Dabei gilt

$$I \in L_D \iff \text{es existiert ein akzeptierender Berechnungspfad von } M \text{ auf } I.$$

Eine Konfiguration enthält insbesondere den aktuellen Zustand, die Position des Kopfes und den Inhalt des Bandes.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Ein Berechnungspfad von M ist eine Folge von Konfigurationen $C_0 \to C_1 \to \cdots \to C_T$, wobei jeder Übergang durch die Übergangsfunktion von M erlaubt ist.

Da M in Polynomialzeit läuft, gibt es ein Polynom p mit $T \leq p(|I|)$ für jeden Berechnungspfad auf Eingabe I.
Dabei gilt

$$I \in L_D \iff \text{es existiert ein akzeptierender Berechnungspfad von } M \text{ auf } I.$$

Eine Konfiguration enthält insbesondere den aktuellen Zustand, die Position des Kopfes und den Inhalt des Bandes.

Idee: Boolesche Variablen beschreiben diese Informationen für jeden Zeitpunkt.
Bspw: Variablen $X_{t,i,a}$ beschreiben, ob zur Zeit t in Bandzelle i das Symbol a steht.
Variablen $Q_{t,q}$ beschreiben, ob M zur Zeit t im Zustand q ist.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Ein Berechnungspfad von M ist eine Folge von Konfigurationen $C_0 \to C_1 \to \cdots \to C_T$, wobei jeder Übergang durch die Übergangsfunktion von M erlaubt ist.

Da M in Polynomialzeit läuft, gibt es ein Polynom p mit $T \leq p(|I|)$ für jeden Berechnungspfad auf Eingabe I.
Dabei gilt

$$I \in L_D \iff \text{es existiert ein akzeptierender Berechnungspfad von } M \text{ auf } I.$$

Eine Konfiguration enthält insbesondere den aktuellen Zustand, die Position des Kopfes und den Inhalt des Bandes.

Idee: Boolesche Variablen beschreiben diese Informationen für jeden Zeitpunkt.
Bspw: Variablen $X_{t,i,a}$ beschreiben, ob zur Zeit t in Bandzelle i das Symbol a steht.
Variablen $Q_{t,q}$ beschreiben, ob M zur Zeit t im Zustand q ist.

Durch geeignete Verknüpfung dieser Variablen stellen wir sicher, dass eine erfüllende Belegung genau dann einen akzeptierenden Berechnungspfad beschreibt, wenn ein solcher existiert.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\mathrm{SAT} \in \mathrm{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \mathrm{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \mathrm{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Ein Berechnungspfad von M ist eine Folge von Konfigurationen $C_0 \to C_1 \to \cdots \to C_T$, wobei jeder Übergang durch die Übergangsfunktion von M erlaubt ist.

Da M in Polynomialzeit läuft, gibt es ein Polynom p mit $T \leq p(|I|)$ für jeden Berechnungspfad auf Eingabe I.
Dabei gilt

$$I \in L_D \iff \text{es existiert ein akzeptierender Berechnungspfad von } M \text{ auf } I.$$

Eine Konfiguration enthält insbesondere den aktuellen Zustand, die Position des Kopfes und den Inhalt des Bandes.

Idee: Boolesche Variablen beschreiben diese Informationen für jeden Zeitpunkt.
Bspw: Variablen $X_{t,i,a}$ beschreiben, ob zur Zeit t in Bandzelle i das Symbol a steht.
Variablen $Q_{t,q}$ beschreiben, ob M zur Zeit t im Zustand q ist.

Durch geeignete Verknüpfung dieser Variablen stellen wir sicher, dass eine erfüllende Belegung genau dann einen akzeptierenden Berechnungspfad beschreibt, wenn ein solcher existiert.

Da es nur polynomial viele Zeitpunkte und relevante Bandzellen gibt und jeder Übergang lokal beschrieben werden kann, kann die Formel φ_I in Polynomialzeit konstruiert werden.

Theorem von Cook-Levin: SAT ist NP-vollständig.

Beweisidee (kompletter Beweis in Berechenbarkeit)

$\text{SAT} \in \text{NP}$ ✓.
SAT NP-schwer?

Sei $D \in \text{NP}$ und sei L_D die Sprache der Ja-Instanzen von D.

Da $D \in \text{NP}$, gibt es eine nicht-determ. TM M, die L_D in Polynomialzeit entscheidet.

Wir müssen für jede beliebige Instanz I von D in Polynomialzeit eine boolesche Formel φ_I konstruieren, sodass

$$\boxed{I \in L_D \iff \varphi_I \text{ ist erfüllbar.}}$$

Hinweis:

$I \in L_D \iff I$ ist JA-instanz von D

Wir müssen φ_I für jede Eingabe I konstruieren können, ohne vorher zu wissen, ob I eine Ja- oder Nein-Instanz ist.

Ein Berechnungspfad von M ist eine Folge von Konfigurationen $C_0 \to C_1 \to \cdots \to C_T$, wobei jeder Übergang durch die Übergangsfunktion von M erlaubt ist.

Da M in Polynomialzeit läuft, gibt es ein Polynom p mit $T \leq p(|I|)$ für jeden Berechnungspfad auf Eingabe I.
Dabei gilt

$$I \in L_D \iff \text{es existiert ein akzeptierender Berechnungspfad von } M \text{ auf } I.$$

Eine Konfiguration enthält insbesondere den aktuellen Zustand, die Position des Kopfes und den Inhalt des Bandes.

Idee: Boolesche Variablen beschreiben diese Informationen für jeden Zeitpunkt.
Bspw: Variablen $X_{t,i,a}$ beschreiben, ob zur Zeit t in Bandzelle i das Symbol a steht.
Variablen $Q_{t,q}$ beschreiben, ob M zur Zeit t im Zustand q ist.

Durch geeignete Verknüpfung dieser Variablen stellen wir sicher, dass eine erfüllende Belegung genau dann einen akzeptierenden Berechnungspfad beschreibt, wenn ein solcher existiert.

Da es nur polynomial viele Zeitpunkte und relevante Bandzellen gibt und jeder Übergang lokal beschrieben werden kann, kann die Formel φ_I in Polynomialzeit konstruiert werden.

$\implies$ SAT NP-schwer. □

NP-vollständigkeit

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

Theorem (Cook-Levin)

SAT *ist NP-vollständig.*

NP-vollständigkeit

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

Theorem (Cook-Levin)

SAT *ist NP-vollständig.*

Damit haben wir ein erstes NP-vollständiges Problem!

NP-vollständigkeit

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

Theorem (Cook-Levin)

SAT *ist NP-vollständig.*

Damit haben wir ein erstes NP-vollständiges Problem!

Inbesondere gilt:

$\Rightarrow$ Jedes Problem $D' \in \mathrm{NP}$ kann auf SAT reduziert werden

$\Rightarrow$ Wenn wir für ein Problem D zeigen können, dass SAT auf D reduziert werden kann
dann ?

NP-vollständigkeit

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

Theorem (Cook-Levin)

SAT *ist NP-vollständig.*

Damit haben wir ein erstes NP-vollständiges Problem!

Inbesondere gilt:

$\Rightarrow$ Jedes Problem $D' \in \mathrm{NP}$ kann auf SAT reduziert werden

$\Rightarrow$ Wenn wir für ein Problem D zeigen können, dass SAT auf D reduziert werden kann dann kann jedes Problem $D' \in \mathrm{NP}$ auf D reduziert werden, da $\preceq_p$ transitiv ist:

$$\forall D' \in \mathrm{NP} \colon D' \preceq_p \mathrm{SAT} \text{ und } \mathrm{SAT} \preceq_p D \implies \forall D' \in NP \colon D' \preceq_p D$$

NP-vollständigkeit

Ein Entscheidungsproblem D ist **NP-vollständig**, wenn

- $D \in \mathrm{NP}$
- D is NP-schwer: jedes Problem in NP kann in (Poly.zeit) auf D reduziert werden kann.
  (symbolisch: $\forall D' \in \mathrm{NP} \colon D' \preceq_p D$)

Theorem (Cook-Levin)

SAT *ist NP-vollständig.*

Damit haben wir ein erstes NP-vollständiges Problem!

Inbesondere gilt:

$\Rightarrow$ Jedes Problem $D' \in \mathrm{NP}$ kann auf SAT reduziert werden

$\Rightarrow$ Wenn wir für ein Problem D zeigen können, dass SAT auf D reduziert werden kann dann kann jedes Problem $D' \in \mathrm{NP}$ auf D reduziert werden, da $\preceq_p$ transitiv ist:

$$\forall D' \in \mathrm{NP} \colon D' \preceq_p \mathrm{SAT} \text{ und } \mathrm{SAT} \preceq_p D \implies \forall D' \in NP \colon D' \preceq_p D$$

Jetzt: NP-vollständigkeit in der Praxis (Tafel).